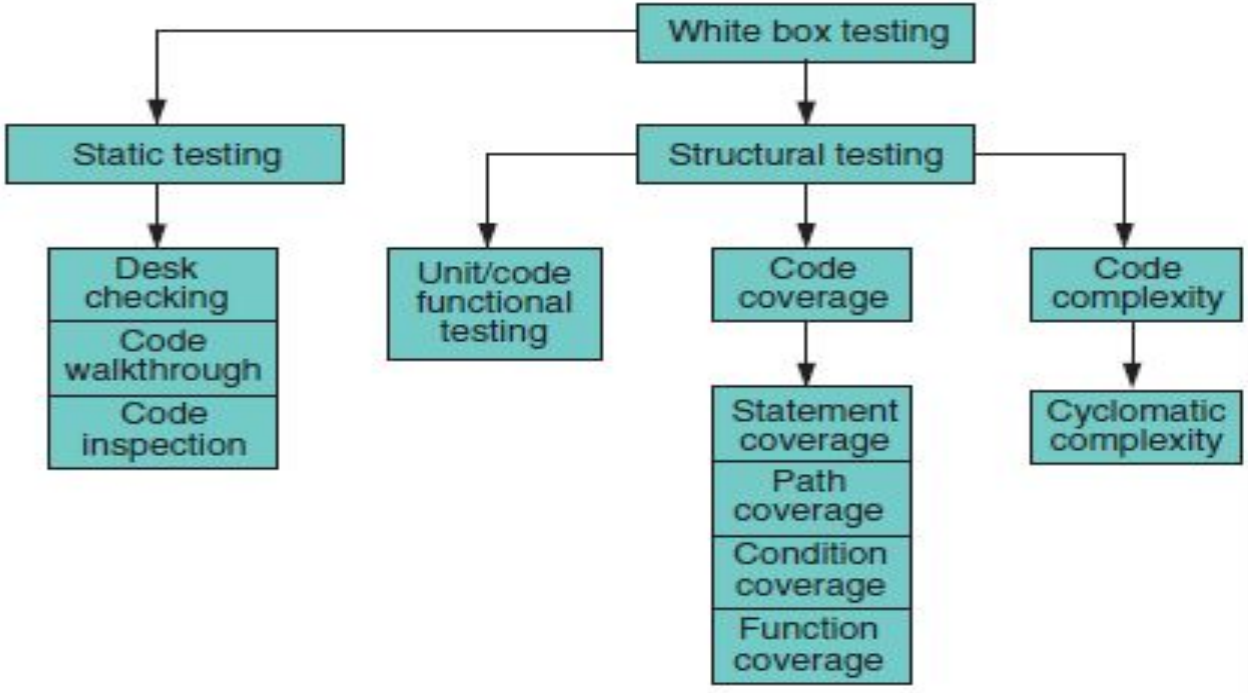
Module - 2 (Testing Techniques and Methodologies)

White box and black box testing techniques: Black box testing - requirements-based testing, boundary value analysis, equivalence partitioning, White box testing - static analysis, unit testing, control flow testing, Integration, system, and acceptance testing methodologies, Non-functional testing techniques - performance, security, and usability testing.

WHAT IS WHITE BOX TESTING?

- **White box testing** is a method of testing where we check the **internal code and logic** of the program to ensure it works correctly.
- White box testing takes into account the **program code, code structure, and internal design flow.**
- The tester examines and tests the **actual program code**.
- Also called clear box testing, glass box testing, or open box testing.
- Defects comes :
  - Defects occur when requirements and design are incorrectly translated into program code.
  - Defects also arise due to programming mistakes and the peculiarities or limitations of programming languages.

Classification of White Box Testing



STATIC TESTING

- Requires only the **source code** of the product, not the binaries or executables.
- It involves **selected people going through the code** to find out whether
  - the **code works** according to the functional requirement;
  - the code has been **written in accordance with the design** developed earlier in the project life cycle;
  - the code for any **functionality has been missed out;**
  - the code **handles errors** properly.
- Static testing can be done by
  - humans
  - with the help of specialized tools.

Static Testing by Humans

- These methods rely on the principle of **humans reading the program code** to detect errors.
- <u>Advantages:</u>
  - Humans can **spot logical mistakes** that computers may miss, like using the wrong variable with a similar name.
  - **Multiple people reviewing code can find more errors** because each person has a different perspective.
  - **Humans can check code against requirements/design** to ensure it does what it is supposed to do.
  - **Many problems and their root causes can be identified and fixed at** once, saving overall fixing time.
  - **Testing before execution saves computer resources,** though it uses human effort.
  - Early detection of defects reduces the cost and time needed to fix them.
  - Finding errors early reduces stress on programmers, preventing rushed fixes and new defects later.

- There are multiple methods to achieve static testing by humans. They are (in the increasing order of formalism) as follows.

1. Desk checking
2. Code walkthrough
3. Code review
4. Code inspection

Desk checking

- Normally done **manually by the author of the code.**
- Verification is done by comparing the code with the **design or specifications** to make sure that the code does what it is supposed to do and effectively.
- Whenever errors are found, the author applies the corrections for errors on the spot.
- This method of catching and correcting errors is characterized by:

  1. No structured method or formalism to ensure completeness and

  2. No maintaining of a log or checklist.

Advantages

- This method relies completely on the author's **thoroughness, diligence, and skills.**
- There is **no process or structure** that guarantees or verifies the effectiveness of desk checking.
- This method is effective for correcting" obvious" coding errors but will not be effective in detecting errors that arise due to incorrect understanding of requirements or incomplete requirements.
- The main advantage offered by this method is that the programmer who knows the code and the programming language very well is well equipped to read and understand his or her own code.
- This is done by one individual, there are fewer scheduling and logistics over-heads.

Disadvantages

- A developer is not the best person to detect problems in his or her own code. He or she may be tunnel visioned and have blind spots to certain types of problems.
- Developers generally prefer to write new code rather than do any form of testing.
- This method is essentially person-dependent and informal and thus may not work consistently across all developers.

Code walkthrough

- group-oriented method
- Walkthroughs are less formal
- It brings multiple perspectives.
- A set of people look at the program code and raise questions for the author.
- The author explains the logic of the code, and answers the questions.
- If the author is unable to answer some questions, he or she then takes those questions and finds their answers.
- Completeness is limited to the area where questions are raised by the team.

Formal inspection

- Code inspection-also called Fagan Inspection -high degree of formalism.
- The focus of this method is to detect all faults, violations, and other side-effects.
- This method increases the number of defects detected by

  1. demanding thorough preparation before an inspection/review;
  2. enlisting multiple diverse views;
  3. assigning specific roles to the multiple participants; and
  4. going sequentially through the code in a structured manner.

When and Who

- A formal inspection should take place only when the author has made sure the code is ready for inspection by performing some basic desk checking and walkthroughs.
- There are four roles in inspection.

  (i) Author of the code.

  (ii) Moderator who is expected to formally run the inspection according to the process.

  (iii) The inspectors: people who actually provides, review comments for the code. There are typically multiple inspectors.

  (iv) Scribe: detailed notes during the inspection meeting and circulates them to the inspection team after the meeting.

How

- The author or the moderator selects the review team.
- The inspectors get copies (These can be hard copies or soft copies) of the code to be inspected along with other supporting documents such as the design document, requirements document, and any documentation of applicable standards.
- The author also presents his or her perspective of what the program is intended to do along with any specific issues that he or she may want the inspection team to put extra focus on.
- The moderator informs the team about the date, time, and venue ofthe inspection meeting.

Process

- The moderator takes the team sequentially through the program code, asking each inspector if there are any defects in that part of the code.
- If any of the inspectors raises a defect, then the inspection team deliberates on the defect .
- Defect is classified into two dimensions-

    (i) minor/major

    (ii) systemic/mis-execution

- Mis-execution Defect -happens because of an error or slip on the part of the author.

  It is unlikely to be repeated later,

- Systemic defects- machine specific errors

  Require correction at a different level.

- Minor defects - defects that may not substantially affect a program
- Major defects - need immediate

Combining various methods

- challenges to watch out for in conducting formal inspections are as follows.

  1. These are time consuming- the process calls for preparation as well as formal meetings, these can take time.

  2. The logistics and scheduling can become an issue since multiple people are involved.

  3. It is not always possible to go through every line of code,

- Identify, during the planning stages, which parts of the code will be subject to formal Inspections.
- Portions of code can be classified on the basis of their criticality or complexity as "high," "medium," and "low."
- High or medium complex critical code should be subject to formal inspections, while those classified as "low" can be subject to either walkthroughs or even desk checking.

CODE REVIEW CHECKLIST

DATA ITEM DECLARATION RELATED

- Are the names of the variables meaningful?
- If the programming language allows mixed case names, are there variable names with confusing use of lower case letters and capital letters?
- Are the variables initialized?
- Are there similar sounding names (especially words in singular and plural)? [These could be possible causes of unintended errors.]
- Are all the common structures, constants, and flags to be used defined in a header file rather than in each file separately?

DATA USAGE RELATED

- Are values of right data types being assigned to the variables?
- Is the access of data from any standard files, repositories, or databases done through publicly supported interfaces?
- If pointers are used, are they initialized properly?
- Are bounds to array subscripts and pointers properly checked?
- Has the usage of similar-looking operators (for example,=and == or & and && in C) checked?

CONTROL FLOW RELATED

- Are all the conditional paths reachable?
- Are all the individual conditions in a complex condition separately evaluated?
- If there is a nested IF statement, are the THEN and ELSE parts appropriately delimited?
- In the case of a multi-way branch like SWITCH / CASE statement, is a default clause provided? Are the breaks after each CASE appropriate?
- Is there any part of code that is unreachable?
- Are there any loops that will never execute?
- Are there any loops where the final condition will never be met and hence cause the program to go into an infinite loop?
- What is the level of nesting of the conditional statements? Can the code be simplified to reduce complexity?

STANDARDS RELATED

- Does the code follow the coding conventions of the organization?
- Does the code follow any coding conventions that are platform specific (for example, GUI calls specific to Windows or Swing)

STYLE RELATED

- Are unhealthy programming constructs (for example, global variables in C, ALTER statement in COBOL) being used in the program?
- Is there usage of specific idiosyncrasies of a particular machine architecture or a given version of an underlying product (for example, using “undocumented” features)?
- Is sufficient attention being paid to readability issues like indentation of code?

MISCELLANEOUS

- Have you checked for memory leaks (for example, memory acquired but not explicitly freed)?

DOCUMENTATION RELATED

- Is the code adequately documented, especially where the logic is complex or the section of code is critical for product functioning?
- Is appropriate change history documented?

Static Analysis Tools

- Reduce the manual work and perform analysis of the code to find out errors such as those listed below.

1. whether there are unreachable codes (usage of GOTO statements sometimes creates this situation; there could be other reasons too)

2. variables declared but not used

3. mismatch in definition and assignment of values to variables

4. illegal or error prone typecasting of variables

5. use of non-portable or architecture-dependent programming constructs

6. memory allocated but not having corresponding statements for freeing them up memory

7. calculation of cyclomatic complexity

Control Flow Testing

- Control Flow Testing is a white-box testing technique that focuses on verifying the control flow of a program.
- It is used to evaluate the logical structure of a program’s control flow (i.e., the flow of execution through the program).
- This type of testing helps identify potential errors or logic flaws in the way decisions, loops, and other control structures are implemented within the code.

Control Flow:

- Refers to the sequence of execution of statements or instructions in a program.
- The flow is determined by constructs like conditions (if, else, switch), loops (for, while), and jumps (goto, break, continue).

Objective of Control Flow Testing:

- Ensure that all paths, decisions, loops, and branches are correctly executed and tested.
- Detect logical errors and unreachable or redundant code.
- Assess the flow of control from one part of the code to another, ensuring correctness.

STRUCTURAL TESTING

- Structural testing takes into account the **code, code structure, internal design, and how they are coded.**
- Run by the computer on the built product
- **Structural testing** means running the actual software using **prepared test cases** to check and cover as much of the **program code** as possible.
- A part of the code is said to be **exercised** when a test case makes the program **execute that specific part of the code** during testing.

Unit/Code Functional Testing

- Some **quick checks** that a developer performs before subjecting the code to more extensive code coverage testing or code complexity testing.
- Quick Checks:
  - Initially, the developer can perform certain obvious tests, knowing the input variables and the corresponding expected output variables. By repeating these tests for multiple values of input variables, the confidence level of the developer to go to the next level. Can be done earlier.
  - For modules with complex logic or conditions, the developer can build a “debug version" of the product by putting intermediate print statements and making sure the program is passing through the right loops and iterations the right number of times.
  - Run the product under a debugger or an Integrated Development Environment (IDE).

Code Coverage Testing

- It involves designing and executing test cases and finding out the percentage of code covered by testing using instrumentation of code.
- Instrumentation does the following :
  - Rebuilds the product, linking the product with a set of libraries.
  - Instrumented code can monitor and find the portions of code covered.
  - Allow reporting on the portions of the code covered frequently.
- Types of coverage
  - Statement coverage
  - Path coverage
  - Condition coverage
  - Function coverage

Statement Coverage

- Statement coverage means designing test cases so that every executable statement in the program is executed at least once.

Statement Coverage for Different Constructs

1. **Sequential Statements**

- Statements execute from top to bottom.
- A single test case is usually enough.
- **But coverage may fail** if:
  - Runtime errors occur (e.g., divide by zero)
  - Code is entered from multiple entry points

2. If–Else Statements

- To cover all statements:
    - One test case must execute the **then** part
    - One test case must execute the **else** part

3. Switch Statements

- Each case must be executed at least once.
- Can be treated as multiple if–else conditions.
- Requires **multiple test cases**

4. Loops

- Loops are error-prone, especially at boundaries.
- Test cases should:

1. **Skip the loop** (condition false initially)
2. **Execute loop normally** (1 to maximum times)
3. **Test boundary values**:
   Just below n
   Exactly n
   Just above n

Statement Coverage Formula

$$\text{Statement Coverage} = \frac{\text{Number of statements executed}}{\text{Total executable statements}} \times 100$$

- More complex statements → more test cases needed
- **100% statement coverage is practically impossible**
- High statement coverage **does NOT guarantee a defect-free program**

Why Statement Coverage Is Not Enough?

Example Problem

```
Total = 0;

if (code == "M") {

   stmt1; stmt2; stmt3; stmt4; stmt5; stmt6; stmt7;

}

else

   percent = value / Total * 100;  // Divide by zero
```

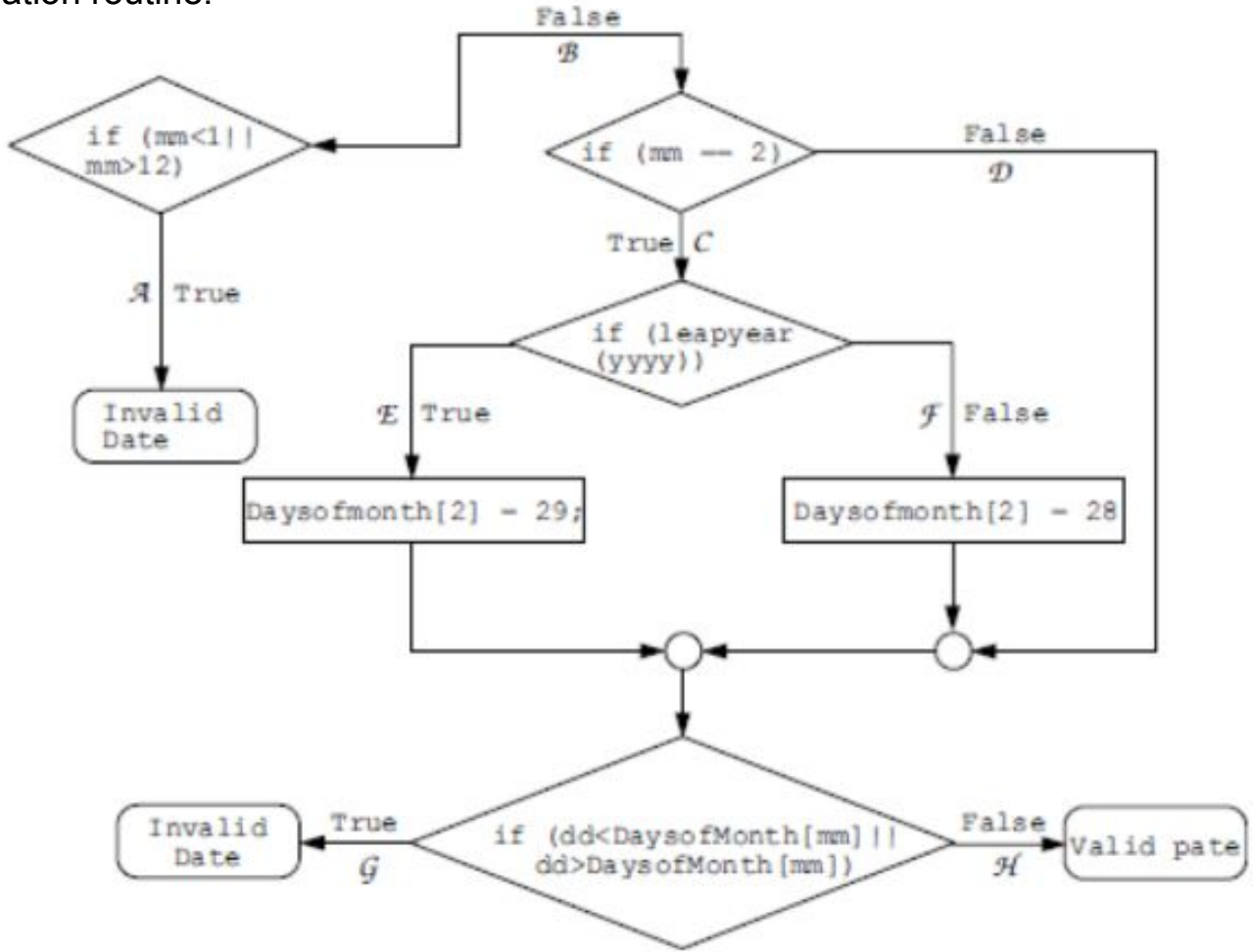
- Testing with code = "M" gives **80% statement coverage**
- But in real usage, if code ≠ "M" most of the time:
    - Program **fails frequently** due to divide-by-zero
- So **high statement coverage can still miss serious defects**

- Statement coverage checks **whether statements run**
- It does **not check all possible execution paths**
- **Path coverage** is needed to overcome these limitations

Path coverage

- In path coverage, we split a program into a number of distinct paths.
- A program (or a part of a program) can start from the beginning and take any of the paths to its completion.
- **Path Coverage = (Total paths exercised / Total number of paths in program) * 100**
- Path coverage provides a stronger condition of coverage than statement coverage as it relates to the various logical paths in the program rather than just program statements.

Date validation routine.



Why Path Testing Is Insufficient

- Some Boolean conditions may never be evaluated
- Logical errors can remain undetected
- Path coverage does not guarantee condition correctness

Consider the condition:

- `if (mm < 1 || mm > 12)`
- If we give `mm = 0`
  Condition `mm < 1` becomes **TRUE**
  The program goes to **Path A (Invalid Month)**
  Path A is covered
- Condition `mm > 12` is **never tested**
- So we cannot be sure that this condition works correctly

Condition coverage

- Each **Boolean condition** must be tested
- Every condition must produce **TRUE and FALSE results.** This ensures proper testing of logic.
- an indication of the percentage of conditions covered by a set of test cases.

Condition Coverage Formula

$$\text{Condition Coverage} = \frac{\text{Number of conditions exercised}}{\text{Total number of conditions in the program}} \times 100$$

Function coverage

- to identify how many program functions are covered by test cases.
- While providing function coverage, test cases can be written so as to exercise each of the different functions in the code.
- Advantages:
  - Functions are easier to identify in a program and hence it is easier to write test cases to provide function coverage.
  - Since functions are at a much higher level of abstraction than code, it is easier to achieve 100 percent function coverage than 100 percent coverage in any of the earlier methods.
  - Functions have a more logical mapping to requirements and hence can provide a more direct correlation to the test coverage of the product.
  - Since functions are a means of realizing requirements, the importance of functions can be prioritized based on the importance of the requirements they realize.
  - Function coverage provides a natural transition to black box testing.
- Help in improving the performance as well as quality of the product.
- **Function Coverage = (Total functions exercised / Total number of functions in program) * 100**

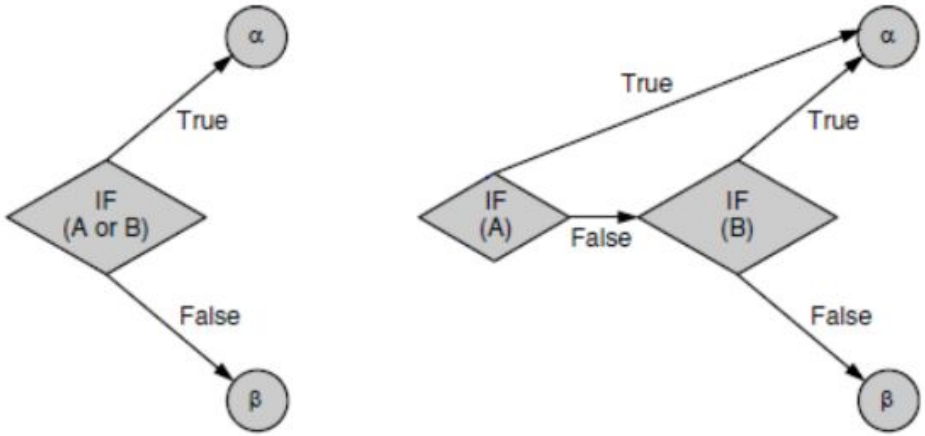
Other uses of code coverage testing other than functional requirements.

- **Performance analysis and optimization**: Code coverage tests can identify the areas of a code that are executed most frequently. Extra attention can then be paid to these sections of the code.
- **Resource usage analysis** : is useful in identifying bottlenecks in resource usage.
- **Checking of critical sections** or concurrency related parts of code : Critical sections are those parts of a code that cannot have multiple processes executing at the same time. Coverage
- **Identifying memory leaks**
- **Dynamically generated code :**White box testing can help identify security holes effectively, especially in a dynamically generated code.

Code Complexity Testing

- **Cyclomatic complexity** is a metric that quantifies the complexity of a program.
- These provides answers to the question:
  - Which of the paths are independent? If two paths are not independent, then we may be able to minimize the number of tests.
  - Is there an upper bound on the number of tests that must be run to ensure that all the statements have been executed at least once?
- A program is represented in the form of a flow graph.
- A flow graph consists of nodes and edges. In order to convert a standard flow chart into a flow graph to compute cyclomatic complexity, the following steps can be taken.

1. Identify the predicates or decision points (typically the Boolean conditions or conditional statements) in the program.
2. Ensure that the predicates are simple. if there are loop constructs, break the loop termination checks into simple predicates.



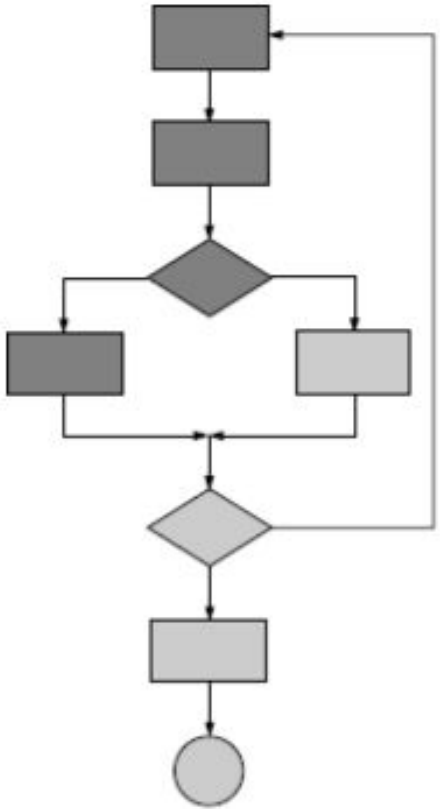
(a) A predicate with a Boolean OR (b) An equivalent set of simple predicates

3. Combine all sequential statements into a single node. The reasoning here is that these statements all get executed, once started.

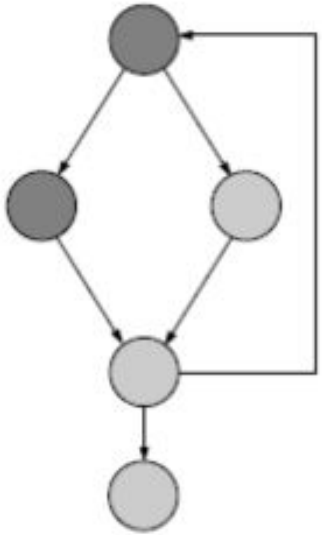
4. When a set of sequential statements are followed by a simple predicate combine all the sequential statements and the predicate check into one node and have two edges emanating from this one node. Such nodes with two edges emanating from them are called predicate nodes.

5. Make sure that all the edges terminate at some node; add a node to represent all the sets of sequential statements at the end of the program.

Conventional flow chart



Flow diagram for calculating complexity



- a flow graph and the cyclomatic complexity provide indicators to the complexity of the logic flow in a program and to the number of independent paths in a program.
- The primary contributors to both the complexity and independent paths are the decision points in the program.

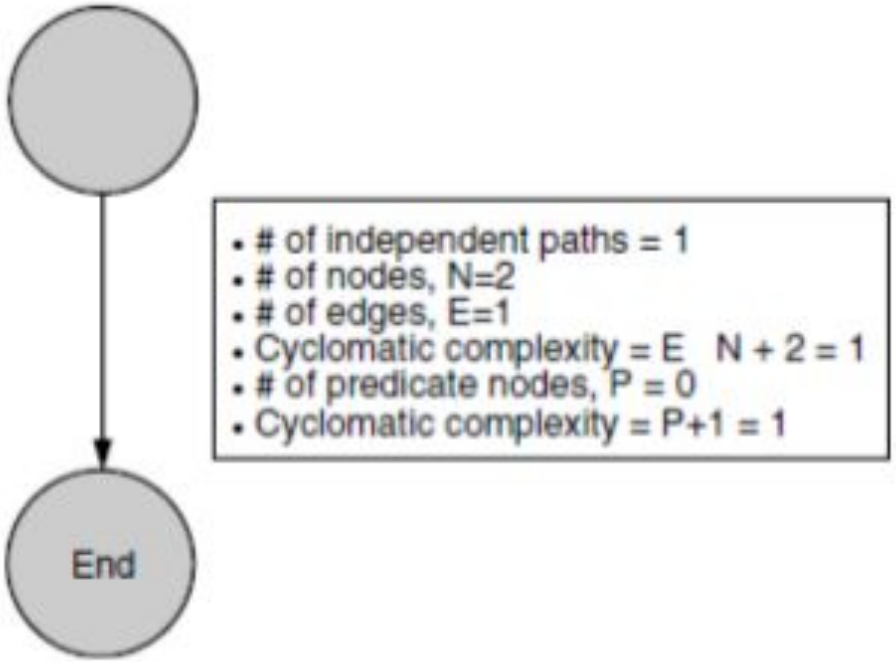
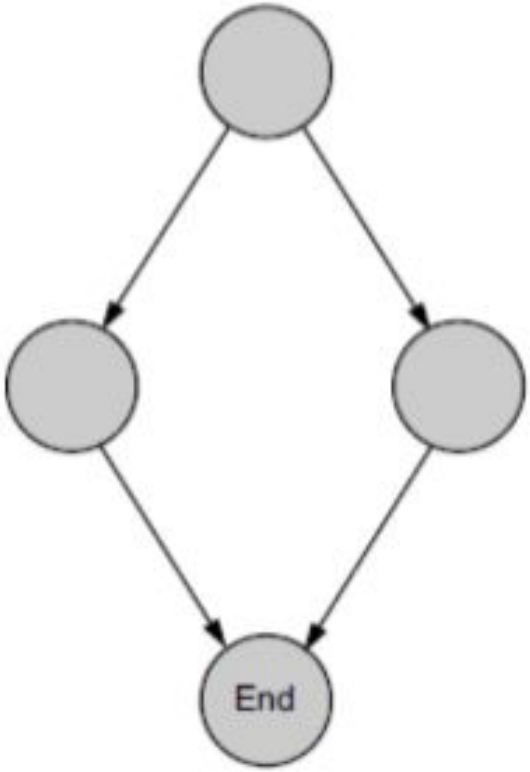


Figure 3.5 A hypothetical program with no decision node.



- # of independent paths = 2
- # of nodes, N=4
- # of edges, E=4
- Cyclomatic complexity = E – N + 2 = 2
- # of predicate nodes, P = 1
- Cyclomatic complexity = P+1 = 2

2 Ways of calculating cyclomatic complexity

Cyclomatic Complexity = Number of Predicate Nodes + 1

- can be used even without drawing the flow graph, by simply counting the number of the basic predicates.

Cyclomatic Complexity = E – N + 2

Calculating and using cyclomatic complexity

- Thus some basic complexity checks must be performed on the modules before embarking upon the testing (or even coding) phase.
- Complexity What it means:

1–10: Well-written code, testability is high, cost/effort to maintain is low

10–20: Moderately complex, testability is medium, cost/effort to maintain is medium

20–40: Very complex, testability is low, cost/effort to maintain is high

>40: Not testable, any amount of money/effort to maintain may not be enough

CHALLENGES IN WHITE BOX TESTING

- White box testing requires a sound knowledge of the program code and the programming language.
- Human tendency of a developer being unable to find the defects in his or her code
- Fully tested code may not correspond to realistic scenarios

Black box testing

- Black box testing involves **looking at the specifications** and does not require examining the **code of a program.**
- Black box testing is done from the **customer's viewpoint.**
- The test engineer engaged in black box testing only knows the set of inputs and expected outputs and is unaware of how those inputs are transformed into outputs by the software.
- Black box testing is done without the knowledge of the internals of the system under test.
- Black box testing thus requires a functional knowledge of the product to be tested.

Eg: a lock and key

- We do not know how the levers in the lock work, but we only know the set of inputs (the number of keys, specific sequence of using the keys and the direction of turn of each key) and the expected outcome (locking and unlocking).
- For example, if a key is turned clockwise it should unlock and if turned anticlockwise it should lock.
- To use the lock one need not understand how the levers inside the lock are constructed or how they work. However, it is essential to know the external functionality of the lock and key system.
- Some of the functionality that you need to know to use the lock are given below.

Functionality	What you need to know to use
Features of a lock	It is made of metal, *has a hole provision to lock,* has a facility to insert the key, and the keyhole ability to turn clockwise or anticlockwise.
Features of a key	It is made of metal and created to fit into a particular lock's keyhole.
Actions performed	Key inserted and turned clockwise to lock Key inserted and turned anticlockwise to unlock
States	Locked Unlocked
Inputs	Key turned clockwise or anticlockwise
Expected outcome	Expected outcome Locking Unlocking

WHY BLACK BOX TESTING

- It helps in the overall **functionality verification** of the system under test.
- Black box testing is done **based on requirements**: It helps in identifying any incomplete, inconsistent requirement as well as any issues involved when the system is tested as a complete entity.
- Black box testing addresses the s**tated requirements as well as implied requirements:** Not all the requirements are stated explicitly, but are deemed implicit. For example, inclusion of dates, page header, and footer may not be explicitly stated in the report generation requirements specification. However, these would need to be included while providing the product to the customer to enable better readability and usability.

- Black box testing encompasses the **end user perspectives**: Since we want to test the behavior of a product from an external perspective, end-user perspectives are an integral part of black box testing.
- Black box testing handles **valid and invalid inputs** : It is natural for users to make errors while using a product. Hence, it is not sufficient for black box testing to simply handle valid inputs. This ensures that the product behaves as expected in a valid situation and does not hang or crash when provided with an invalid input. These are called positive and negative test cases.

WHEN TO DO BLACK BOX TESTING?

- Black box testing activities require involvement of the **testing team from the beginning of the software project life cycle,** regardless of the software development life cycle model chosen for the project.
- **Testers** can get involved right from the **requirements gathering and analysis** phase for the system under test.
- **Test scenarios and test data** are prepared during the **test construction phase** of the test life cycle, when the software is in the design phase.
- Once the code is ready and delivered for testing, test execution can be done.
- All the test scenarios developed during the construction phase are executed.

HOW TO DO BLACK BOX TESTING?

1. **Requirements based testing**
2. Positive and negative testing
3. **Boundary value analysis**
4. Decision tables
5. **Equivalence partitioning**
6. State based testing
7. Compatibility testing
8. User documentation testing
9. Domain testing

1. Requirements Based Testing

- Requirements testing deals with **validating the requirements** given in the **Software Requirements Specification (SRS)** of the software system.
- Explicit requirements are stated and documented as part of the requirements specification.
- Implied or implicit requirements are those that are nor documented but assumed to be incorporated in the system.
- The precondition for requirements testing is a detailed review of the requirements specification.
- Requirements review ensures that they are consistent, correct, complete, and testable.
- All explicit requirements (from the Systems Requirements Specifications) and implied requirements (inferred by the test team) are collected and documented as **“Test Requirements Specification” (TRS).**

Sample requirements specification for lock and key system.

S.No.	Requirements identifier	Description	Priority (High, med, low)
1	BR–01	Inserting the key numbered 123–456 and turning it clockwise should facilitate locking	H
2	BR–02	Inserting the key numbered 123–456 and turning it anticlockwise should facilitate unlocking	H
3	BR–03	Only key number 123–456 can be used to lock and unlock	H
4	BR–04	No other object can be used to lock	M
5	BR–05	No other object can be used to unlock	M
6	BR–06	The lock must not open even when it is hit with a heavy object	M
7	BR–07	The lock and key must be made of metal and must weigh approximately 150 grams	L
8	BR–08	Lock and unlock directions should be changeable for usability of left-handers	L

- Each **requirement is given a unique id along with a brief description.**
- The requirement identifier and description can be taken from the requirements Specification or any other available document that lists the requirements to be tested for the product.
- the naming convention uses a prefix “BR” followed by a two-digit number. BR indicates the type of testing—"Black box-requirements testing.”
- The two-digit numerals count the number of requirements.
- In systems that are more complex, an identifier representing a module and a running serial number within the module (for example, INV-01, AP-02, and so on) can identify a requirement.
- Each requirement is assigned a requirement priority, classified as high, medium or low. This not only enables prioritizing the resources for development of features but is also used to sequence and run tests.

- Requirements are tracked by a **Requirements Traceability Matrix (RTM).**
- An RTM traces all the requirements from their genesis through design, development, and testing.
- This matrix evolves through the life cycle of the project.
- The “test conditions” column lists the different ways of testing the requirement.
- The “test case IDs” column can be used to complete the mapping between test cases and the requirement.

Table 4.2 Sample Requirements Traceability Matrix.

Req. ID	Description	Priority (H,M,L)	Test conditions	Test case IDs	Phase of testing
BR–01	Inserting the key numbered 123–456 and turning it clockwise should facilitate locking	H	* Use key 123–456	Lock_001	Unit, Component
BR–02	Inserting the key numbered 123–456 and turning it anticlockwise should facilitate unlocking	H	* Use key 123–456	Lock_002	Unit, Component
BR–03	Only key number 123–456 can be used to lock and unlock	H	* Use key 123–456 to lock * Use key 123–456 to unlock	Lock_003 Lock_004	Component
BR–04	No other object can be used to lock	M	* Use key 789–001 * Use hairpin * Use screwdriver	Lock_005 Lock_006 Lock_007	Integration
BR–05	No other object can be used to unlock	M	* Use key 789–001 * Use hairpin * Use screwdriver	Lock_008 Lock_009 Lock_010	Integration
BR–07	The lock and key must be made of metal and must weigh approximately 150 grams	L	* Use weighing machine	Lock_012	System
BR–08	Lock and unlock directions changed for usability of lefthanders	L			Not implemented

- Once the test case creation is completed, the RTM helps in identifying the relationship between the requirements and test cases.
- The following combinations are possible.
    - One to one—For each requirement there is one test case (for example, BR-01)
    - One to many—For each requirement there are many test cases (for example, BR-03)
    - Many to one—A set of requirements can be tested by one test case (not represented in Table 4.2)
    - Many to many—Many requirements can be tested by many test cases (these kind of test cases are normal with integration and system testing; however, an RTM is not meant for this purpose)
    - One to none—The set of requirements can have no test cases.
- A requirement is subjected to multiple phases of testing—unit, component, integration, and system testing.

An RTM plays a valuable role in requirements based testing.

1. Regardless of the number of requirements, ideally each of the r**equirements has to be teste**d. When there are a large numbers of requirements, it would not be possible for someone to **manually keep a track of the testing status** of each requirement. The RTM provides a **tool to track the testing status** of each requirement, without missing any (key) requirements.

2. By **prioritizing the requirements**, the RTM enables **testers to prioritize the test cases** execution to catch defects in the high-priority area as early as possible. It is also used to find out whether there are adequate test cases for high-priority requirements and to reduce the number of test cases for low- priority requirements. In addition, if there is a crunch time for testing, the prioritization enables s**electing the right features to test.**

3. Test conditions can be **grouped to create test cases or can be represented as unique test cases.** The list of test case(s) that address a particular requirement can be viewed from the RTM.

4. Test conditions/cases can be used as inputs to arrive at a size / effort / schedule estimation of tests.

The **Requirements Traceability Matrix** provides a wealth of information on various test metrics. Some of the **metrics that can be collected or inferred** from this matrix are as follows.

- **Requirements addressed priority wise**—This metric helps in knowing the test coverage based on the requirements. Number of tests that is covered for high-priority requirement versus tests created for low-priority requirement.
- N**umber of test cases requirement wis**e—For each requirement, the total number of test cases created.
- **Total number of test cases prepared**—Total of all the test cases prepared for all requirements.

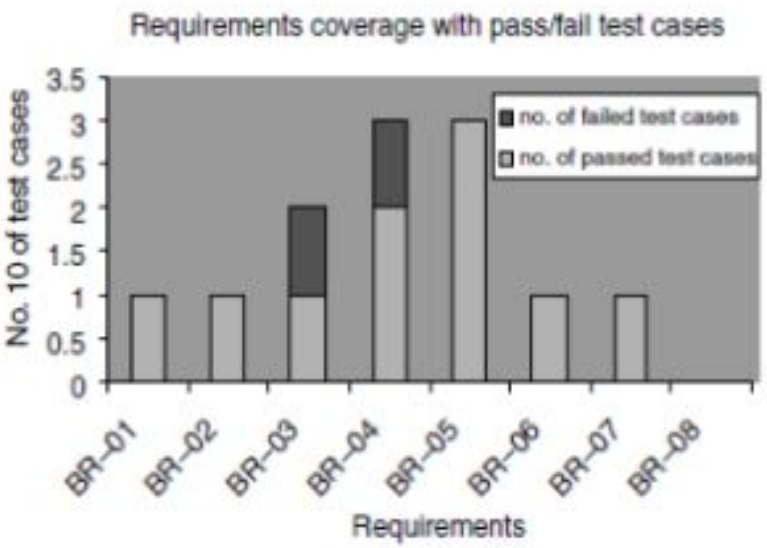
- Once the test cases are **executed**, the test results can be used to collect metrics such as :
- Total number of **test cases (or requirements) passed**—Once test execution is completed, the total passed test cases and what percent of requirements they correspond.
- Total number of **test cases (or requirements) failed**—Once test execution is completed, the total number of failed test cases and what percent of requirements they correspond.
- Total number of **defects in requirements**—List of defects reported for each requirement (defect density for requirements). This helps in doing an impact analysis of what requirements have more defects and how they will impact customers. A comparatively high-defect density in low-priority requirements is acceptable for a release. A high-defect density in high-priority requirement is considered a high-risk area, and may prevent a product release.
- **Number of requirements completed**—Total number of requirements successfully completed without any defects.
- **Number of requirements pending**—Number of requirements that are pending due to defects.

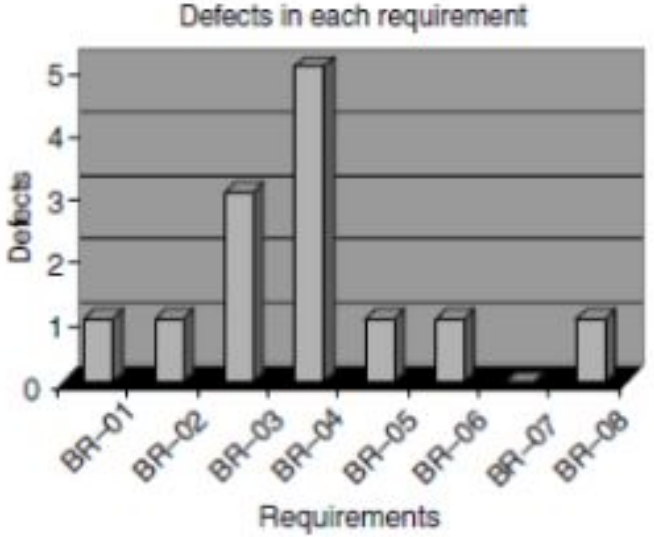
Table 4.3 Sample test execution data.

S.No.	Req. ID	Priority	Test cases	Total test cases	Test cases passed	Test cases failed	% Pass	No. of defects
1	BR–01	H	Lock_01	1	1	0	100	1
2	BR–02	H	Lock_02	1	1	0	100	1
3	BR–03	H	Lock_03, 04	2	1	1	50	3
4	BR–04	M	Lock_05, 06, 07	3	2	1	67	5
5	BR–05	M	Lock_08, 09, 10	3	3	0	100	1
6	BR–06	L	Lock_11	1	1	0	100	1
7	BR–07	L	Lock_12	1	1	0	100	0
8	BR–08	L		0	0	0	0	1
Total	8			**12**	**10**	2	83	**12**

From the above table, the following observations can be made with respect to the requirements.

- 83 percent passed test cases correspond to 71 percent of requirements being met (five out of seven requirements met; one requirement not implemented). Similarly, from the failed test cases, outstanding defects affect 29 percent (= 100 - 71) of the requirements
- There is a high-priority requirement, BR-03, which has failed. There are three corresponding defects that need to be looked into and some of them to be fixed and test case Lock_04 need to be executed again for meeting this requirement. Please note that not all the three defects may need to be fixed, as some of them could be cosmetic or low-impact defects.
- There is a medium-priority requirement, BR-04, has failed. Test case Lock_06 has to be re-executed after the defects (five of them) corresponding to this requirement are fixed.
- The requirement BR-08 is not met; however, this can be ignored for the release, even though there is a defect outstanding on this requirement, due to the low-priority nature of this requirement.


Requirements coverage with pass/fail test cases
No. 10 of test cases
3.5
3
2.5
2
1.5
1
0.5
0
no. of failed test cases
no. of passed test cases
BR-01
BR-02
BR-03
BR-04
BR-05
BR-06
BR-07
BR-08
Requirements


Defects in each requirement
Defects
5
4
3
2
1
0
BR-01
BR-02
BR-03
BR-04
BR-05
BR-06
BR-07
BR-08
Requirements

Boundary Value Analysis

- **Conditions and boundaries** are two major sources of defects in a software product.
- By conditions, we mean situations wherein, based on the values of various variables, certain actions would have to be taken.
- By boundaries, we mean “limits” of values of the various variables.
- Boundary value analysis (BVA), a method useful for arriving at tests that are effective in catching defects that happen at boundaries.
- Boundary value analysis believes and extends the concept that the density of defect is more towards the boundaries.
- Boundary value analysis is useful to generate test cases when the input (or output) data is made up of clearly identifiable boundaries or ranges.
- **boundary value testing is extremely useful in uncovering defects is when there are internal limits placed on certain resources, variables, or data structures.**

Number of units bought	Price per unit
First ten units (that is, from 1 to 10 units)	$5.00
Next ten units (that is, from units 11 to 20 units)	$4.75
Next ten units (that is, from units 21 to 30 units)	$4.50
More than 30 units	$4.00

To illustrate the concept of errors that happen at boundaries, let us consider a billing system that offers volume discounts to customers.

- Let us consider a hypothetical store that sells certain commodities and offers different pricing for people buying in different quantities—that is, priced in different “slabs.”
- if we buy 5 units, we pay 5*5 = $25. If we buy 11 units, we pay 5*10 = $50 for the first ten units and $4.75 for the eleventh item. Similarly, if we buy 15 units, we will pay 10*5 + 5*4.75 = $73.75.

The question from a testing perspective for the above problem is what test data is likely to reveal the most number of defects in the program?

Generally it has been found that most defects in situations such as this happen around the boundaries—for example, when buying 9, 10, 11, 19, 20, 21, 29, 30, 31, and similar number of items. While the reasons for this phenomenon is not entirely clear, some possible reasons are as follows.

- Programmers' tentativeness in using the right comparison operator, for example, whether to use the **< = operator or < operator** when trying to make comparisons.
- Confusion caused by the **availability of multiple ways to implement loops and condition checking**. For example, in a programming language like C, we have for loops, while loops and repeat loops. Each of these have different terminating conditions for the loop and this could cause some confusion in deciding which operator to use, thus skewing the defects around the boundary conditions.
- The requirements themselves **may not be clearly understood**, especially around the boundaries, thus causing even the correctly coded program to not perform the correct way.

tests that should be performed and the expected values of the output variable (the cost of the units ordered).This table only includes the positive test cases.

Table 4.6 Boundary values for the volumes discussed in the example.

Value to be tested	Why this value should be tested	Expected value of the output
1	The beginning of the first slab	$5.00
5	A value in the first slab, removed from the boundaries	$25.00
9	Just below the second slab or just at the end of the first slab	$45.00
10	The limit for the second slab	$50.00
11	Just above the first slab, just into the second slab	$54.75
16	A value in the second slab, removed from the boundaries	$28.50
19	Just below the third slab or just at the end of the second slab	$92.75
20	The limit for the third slab	$97.50
21	Just above the third slab, just into the third slab	$102.00
27	A value in the third slab, removed from the boundaries	$129.00
29	Just below the fourth slab or just at the end of the third slab	$138.00
30	The limit for the fourth slab	$142.50
31	Just above the fourth slab	$146.50
50	Well above the lower limit for the fourth slab	$182.50

To summarize boundary value testing.

- Look for any kind of **gradation or discontinuity in data values** which affect computation—the discontinuities are the boundary values, which require thorough testing.
- Look for any **internal limits such as limits on resource**s (as in the example of buffers given above). The behavior of the product at these limits should also be the subject of boundary value testing.
- Also include in the **list of boundary values,** documented limits on hardware resources. For example, if it is documented that a product will run with minimum 4MB of RAM, make sure you include test cases for the minimum RAM (4MB in this case).
- The examples given above discuss boundary conditions for input data—the same analysis needs to be done for output variables also.

Equivalence Partitioning

- Equivalence partitioning is a software testing technique that involves identifying a **small set of representative input values that produce as many different output conditions as possible.**
- This reduces the **number of permutations and combinations of input, output values used for testing,** thereby increasing the coverage and reducing the effort involved in testing.
- The set of input values that generate one single expected output is called a **partition.**
- When the behavior of the software is the same for a set of values, then the set is termed as an **equivalence class or a partition**.

- In this case, one r**epresentative sample from each partition** (also called the member of the equivalance class) is picked up for **testing.**
- One sample from the partition is enough for testing as the result of picking up some more values from the set will be the same and will not yield any additional defects. Since all the values produce equal and same output they are termed as equivalance partition.
- Testing by this technique involve

  (a) identifying all partitions for the complete set of input, output values for a product

  (b) picking up one member value from each partition for testing to maximize complete coverage.

- From the results obtained for a member of an equivalence class or partition, this technique extrapolates the expected results for all the values in that partition.
- The advantage of using this technique is that we gain good coverage with a small number of test cases.
- For example, if there is a defect in one value in a partition, then it can be extrapolated to all the values of that particular partition.
- By using this technique, redundancy of tests is minimized by not repeating the same tests for multiple values in the same partition.

Let us consider the example below, of an insurance company that has the following premium rates based on the age group.

Life Insurance Premium Rates:

A life insurance company has base premium of $0.50 for all ages. Based on the age group, an additional monthly premium has to be paid. For example, a person aged 34 has to pay a premium=base premium + additional premium=$0.50 + $1.65=$2.15.

Age group	Additional premium
Under 35	$1.65
35-59	$2.87
60+	$6.00

Based on the equivalence partitioning technique, the equivalence partitions that are based on age are given below:

- Below 35 years of age (valid input)
- Between 35 and 59 years of age (valid input)
- Above 60 years of age (valid input)
- Negative age (invalid input)
- Age as 0 (invalid input)
- Age as any three-digit number (valid input)

- We need to pick up representative values from each of the above partitions. You may have observed that even though we have only a small table of valid values, the equivalence classes should also include samples of invalid inputs. This is required so that these invalid values do not cause unforeseen errors.
- **Table 4.8** Equivalence classes for the life insurance premium example.

S.No.	Equivalence partitions	Type of input	Test data	Expected results
1.	Age below 35	Valid	26, 12	Monthly premium = $(0.50 + 1.65) = $2.15
2.	Age 35–59	Valid	37	Monthly premium = $(0.50 + 2.87) = $3.37
3.	Age above 60	Valid	65, 90	Monthly premium = $(0.50 + 6.0) = $6.5
4.	Negative age	Invalid	–23	Warning message—Invalid input
5.	Age as 0	Invalid	0	Warning message—Invalid input

- Each row is taken as a single test case and is executed.
- For example, when a person's age is 48, row number 2 test case is applied and the expected result is a monthly premium of $3.37. Similarly, in case a person's age is given as a negative value, a warning message is displayed, informing the user about the invalid input.

The steps to prepare an equivalence partitions table are as follows.

- Choose criteria for doing the equivalence partitioning (range, list of values, and so on)
- Identify the valid equivalence classes based on the above criteria (number of ranges allowed values, and so on)
- Select a sample data from that partition.
- Write the expected result based on the requirements given
- Identify special values, if any, and include them in the table
- Check to have expected results for all the cases prepared
- If the expected result is not clear for any particular test case, mark appropriately and escalate for corrective actions. If you cannot answer a question, or find an inappropriate answer, consider whether you want to record this issue on your log and clarify with the team that arbitrates/dictates the requirements.

Integration testing

- A system is made up of multiple components or modules that can comprise hardware and software.
- Integration is defined as the set of interactions among components.
- Testing the interaction between the modules and interaction with other systems externally is called integration testing.
- Integration testing starts when two of the product components are available and ends when all component interfaces have been tested.
- The final round of integration involving all components is called Final Integration Testing (FIT), or system integration.
- Integration testing is both a type of testing and a phase of testing.

INTEGRATION TESTING AS A TYPE OF TESTING

- Integration testing means testing of interfaces.
- Two types:

(i) internal interfaces : provide communication across two modules within a project or product, internal to the product, and not exposed to the customer or external developers.

(ii) exported or external interfaces: visible outside the product to third party developers and solution providers.

Application Programming Interfaces (APIs)

- One of the methods of achieving interfaces.
- APIs enable one module to call another module.
- The calling module can be internal or external.

other means of integration among the various modules:

- simple function calls, public functions
- facets of programming language constructs like global variables
- operating system constructs like semaphares and shared memory.

stub

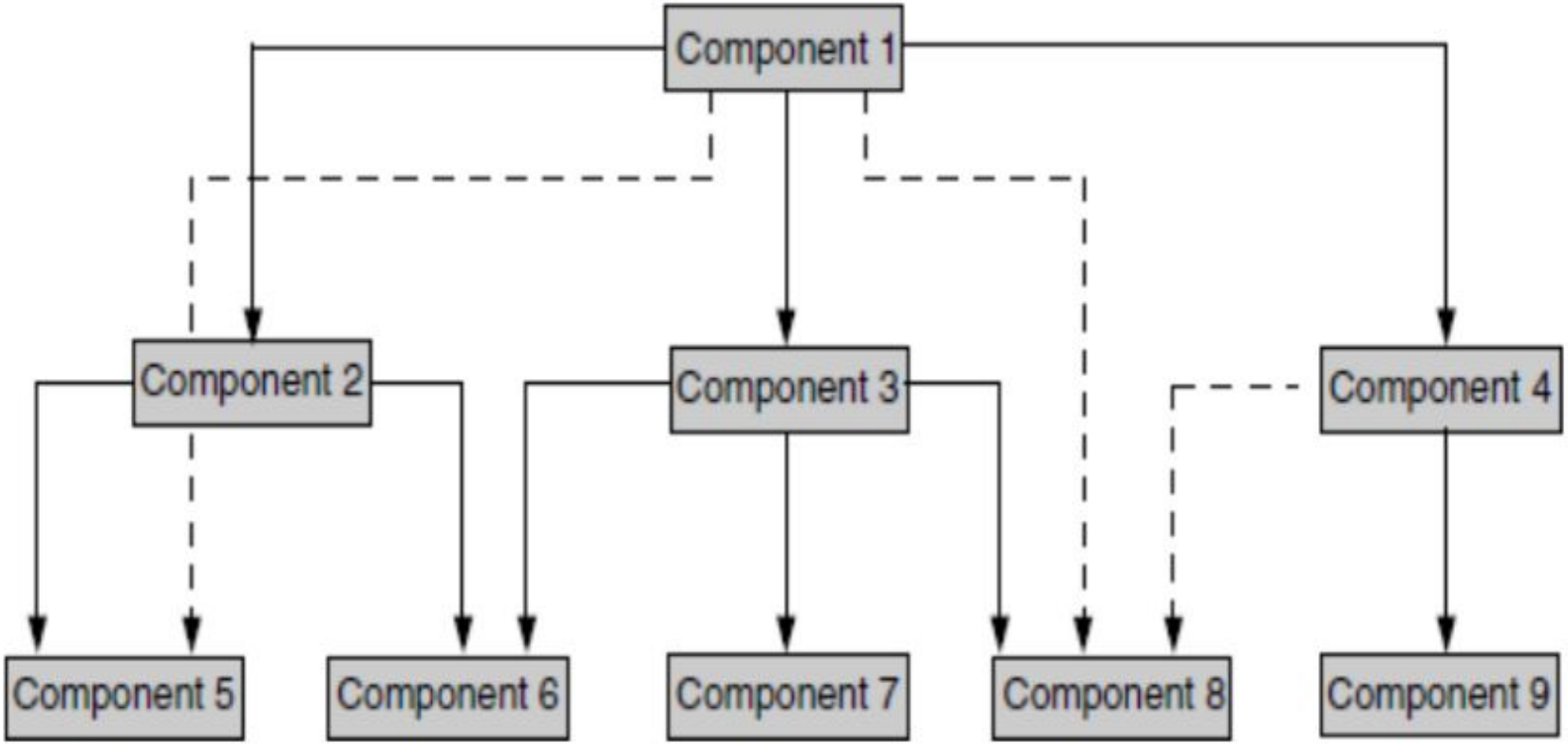
- In order to test the interfaces, when the full functionality of the component being introduced is not available, stubs are provided.
- A stub simulates the interface by providing the appropriate values in the appropriate format as would be provided by the actual component being integrated.
-

classification of interfaces

Explicit interfaces: are documented interfaces

Implicit interfaces: are those which are known internally to the software engineers but are not documented.

- The testing (white box/black box) should look for both implicit and explicit interfaces and test all those interactions.



solid lines represent explicit interfaces
dotted lines represent implicit interfaces

There are several methodologies available, to in decide the order for integration testing. These are as follows.

1. Top-down integration

2. Bottom-up integration

3. Bi-directional integration

4. System integration

1. Top-Down Integration

- Integration testing involves testing the topmost component interface with other components in same order as you navigate from top to bottom, till you cover all the components.
- If a set of components and their related interfaces can deliver functionality without expecting the presence of other components or with minimal interface requirement in the software/product, then that set of components and their related interfaces is called as a “sub-system.”
- Each sub-system in a product can work independently with or without other sub-systems.
- This approach reflects the Waterfall or V model of software development.
- If a component at a higher level requires a modification every time a module gets added to the bottom, then for each component addition integration testing needs to be repeated starting from step 1.

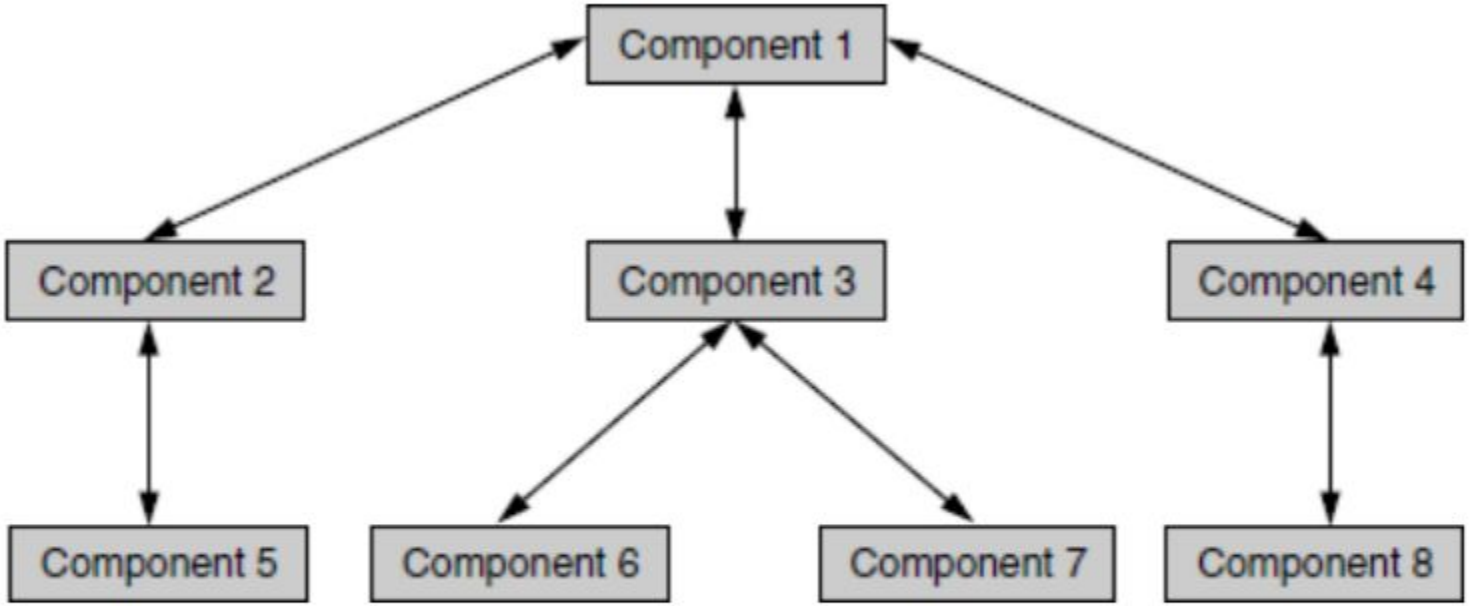
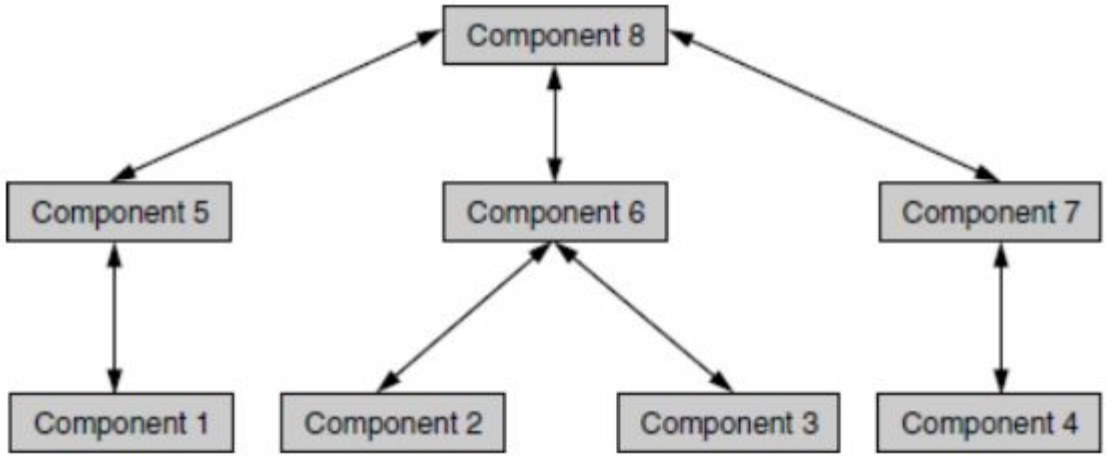


Figure 5.2 Example of top down integrations.

Step	Interfaces tested
1	1-2
2	1-3
3	1-4
4	1-2-5
5	1-3-6
6	1-3-6-(3-7)
7	(1-2-5)-(1-3-6-(3-7))
8	1-4-8
9	(1-2-5)-(1-3-6-(3-7))-(1-4-8)

2. Bottom-up Integration

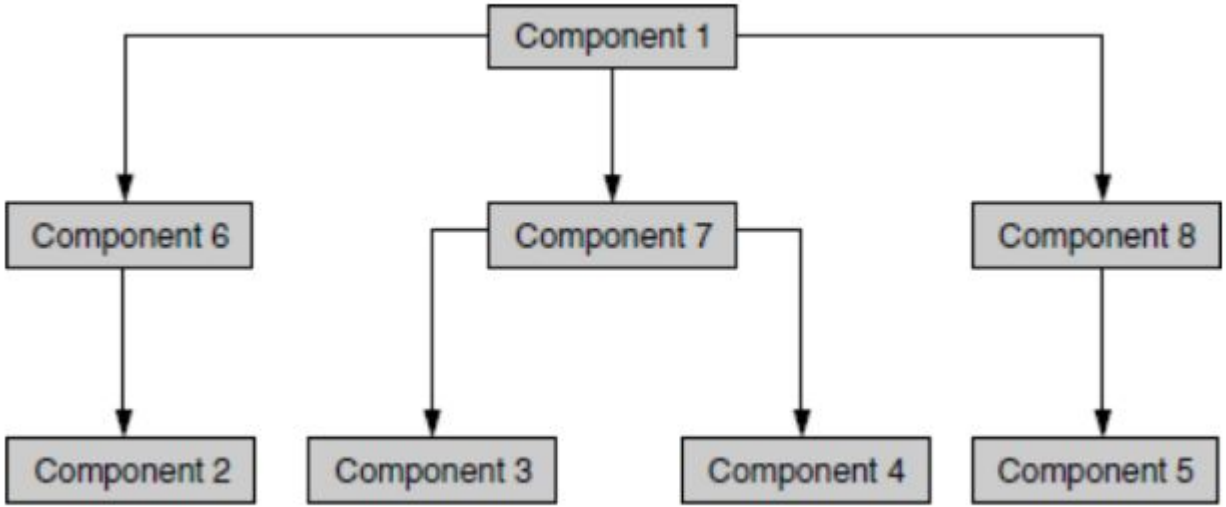
- the components for a new product development become available in reverse order, starting from the bottom.
- bottom-up approach for the iterative and agile methodologies.



Step	Interfaces tested
1	1-5
2	2-6, 3-6
3	2-6-(3-6)
4	4-7
5	1-5-8
6	2-6-(3-6)-8
7	4-7-8
8	(1-5-8)-(2-6-(3-6)-8)-(4-7-8)

3. Bi-Directional Integration

- Bi-directional integration is a combination of the top-down and bottom-up integration approaches used together to derive integration steps.



Step	Interfaces tested
1	6-2
2	7-3-4
3	8-5
4	(1-6-2)-(1-7-3-4)-(1-8-5)

- The individual components 1, 2, 3, 4, and 5 are tested separately and bi-directional integration is performed initially with the use of stubs and drivers.
- Drivers are used to provide upstream connectivity while stubs provide downstream connectivity.
- A driver is a function which redirects the requests to some other component and stubs simulate the behavior of a missing component.
- After the functionality of these integrated components are tested, the drivers and stubs are discarded.
- Once components 6, 7, and 8 become available, the integration methodology then focuses only on those components, as these are the components which need focus and are new. This approach is also called “sandwich integration.”
- An area where this approach comes in handy is when migrating from a two-tier to a three-tier environment.
- In the product development phase when a transition happens from two-tier architecture to three-tier
- architecture, the middle tier (components 6–8) gets created as a set of new components from the code taken from bottom-level applications and top-level services.

4. System Integration

- System integration means that all the components of the system are integrated and tested as a single unit.
- Integration testing, which is testing of interfaces, can be divided into two types:
  - Components or sub-system integration
  - Final integration testing or system integration
- The salient point this testing methodology raises, is that of optimization.
- Instead of integrating component by component and testing, this approach waits till all components arrive and one round of integration testing is done. This approach is also called big-bang integration. It reduces testing effort and removes duplication in testing.

Big bang integration

- It is ideal for a product where the interfaces are stable with less number of defects.
- System integration using the big bang approach is well suited in a product development scenario where the majority of components are already available and stable and very few components get added or modified.

disadvantages

- When a failure or defect is encountered during system integration, it is very difficult to locate the problem, to find out in which interface the defect exists. The debug cycle may involve focusing on specific interfaces and testing them again.
- The ownership for correcting the root cause of the defect may be a difficult issue to pinpoint.
- When integration testing happens in the end, the pressure from the approaching release date is very high. This pressure on the engineers may cause them to compromise on the quality of the product.
- A certain component may take an excessive amount of time to be ready. This precludes testing other interfaces and wastes time till the end.

Guidelines on selection of integration method.

1 Clear requirements and design :Top-down

2 Dynamically changing requirements, design, architecture: Bottom-up

3 Changing architecture, stable design :Bi-directional

4 Limited changes to existing architecture with less impact: Big bang

5 Combination of above :Select one of the above after careful analysis

SYSTEM TESTING

- System testing is defined as a testing phase conducted on the complete integrated system, to evaluate the system compliance with its specified requirements.
- It is done after unit, component, and integration testing phases.
- The testing conducted on the complete integrated products and solutions to evaluate system compliance with specified requirements on functional and nonfunctional aspects is called system testing.
- A system is a complete set of integrated components that together deliver product functionality and features.
- A system can also be defined as a set of hardware, software, and other parts that together provide product features and solution.
- System testing helps in uncovering the defects that may not be directly attributable to a module or an interface.
- System testing brings out issues that are fundamental to design, architecture, and code of the whole product.

- System testing is the only phase of testing which tests the both functional and non-functional aspects of the product.
- the functional side: system testing focuses on real-life customer usage of the product and solutions. System testing simulates customer deployments. For a general-purpose product, system testing also means testing it for different business verticals and applicable domains such as insurance, banking, asset Management, etc

non-functional side, system brings in different testing types (also called quality factors):

1. Performance/Load testing: To evaluate the time taken or response time of the system to perform its required functions in comparison with different versions of same product(s) or a different competitive product(s) is called performance testing.

2. Scalability testing :A testing that requires enormous amount of resource to find out the maximum capability of the system parameters is called scalability testing.

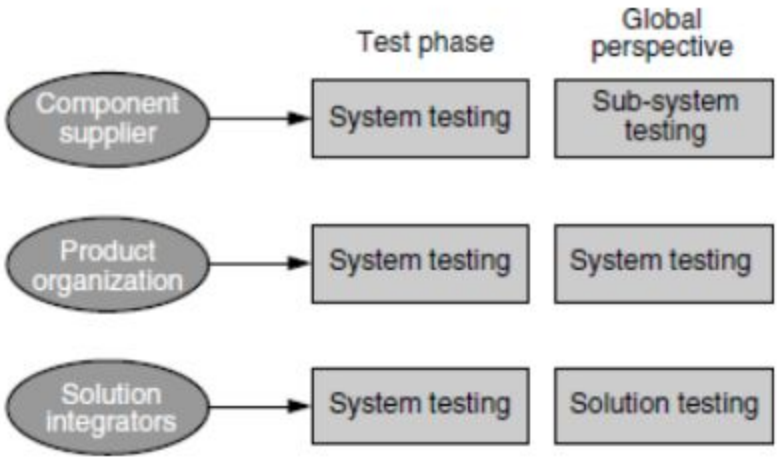
3. Reliability testing :To evaluate the ability of the system or an independent component of the system to perform its required functions repeatedly for a specified period of time is called reliability testing.

4. Stress testing: Evaluating a system beyond the limits of the specified requirements or system resources (such as disk space, memory, processor utilization) to ensure the system does not break down unexpectedly is called stress testing.

5. Interoperability testing: This testing is done to ensure that two or more products can exchange information, use the information, and work closely.

6. Localization testing Testing conducted to verify that the localized product works in different languages is called localization testing.

system testing performed by various organizations from their own as well as from a global perspective.



- System testing is performed on the basis of written test cases according to information collected from detailed architecture/design documents, module specifications, and system requirements specifications.
- System test cases are created after looking at component and integration test cases, and are also at the same time designed to include the functionality that tests the system together.
- System test cases can also be developed based on user stories, customer discussions, and points made by observing typical customer usage.
- System testing may not include many negative scenario verification, such as testing for incorrect and negative values. This is because such negative testing would have been already performed by component and integration testing and may not reflect real-life customer usage.
- System testing may be started once unit, component, and integration testing are completed. This would ensure that the more basic program logic errors and defects have been corrected.

WHY IS SYSTEM TESTING DONE?

- The behavior of the complete product is verified during system testing.
- Tests that refer to multiple modules, programs, and functionality are included in system testing.
- Testing the complete product behavior is critical as it is wrong to believe that individually tested components will work together when they are put together.
- System testing helps in identifying as many defects as possible before the customer finds them in the deployment.
- This is the last chance for the test team to find any remaining product defects before the product is handed over to the customer.\
- an impact analysis is done for those defects to reduce the risk of releasing a product with defects. If the risk of the customers getting exposed to the defects is high, then the defects are fixed before the release; else, the product is released as such.
-

system testing is done for the following reasons.

1. Provide independent perspective in testing
2. Bring in customer perspective in testing
3. Provide a “fresh pair of eyes” to discover defects not found earlier by testing
4. Test product behavior in a holistic, complete, and realistic environment
5. Test both functional and non-functional aspects of the product
6. Build confidence in the product
7. Analyze and reduce the risk of releasing the product
8. Ensure all requirements are met and ready the product for acceptance testing.

ACCEPTANCE TESTING

- Acceptance testing is a phase after system testing that is normally done by the customers or representatives of the customer.
- The customer defines a set of test cases that will be executed to qualify and accept the product.
- These test cases are executed by the customers themselves to quickly judge the quality of the product before deciding to buy the product.
- Acceptance test cases are normally small in number and are not written with the intention of finding defects.
- the acceptance tests are performed by the product organization alone, acceptance tests are executed to verify if the product meets the acceptance criteria defined during the requirements definition phase of the project.
- Acceptance test cases failing in a customer site may cause the product to be rejected and may mean financial loss or may mean rework of product involving effort and time.

Types of Acceptance Testing:

Alpha Testing: Performed by developers or testers in the development environment before release to the client.

Beta Testing: Conducted by a limited number of end users outside the organization who use the system in a real-world environment.

Contract Acceptance Testing: Ensures that the system complies with the terms of the contract, including specific performance criteria.

Regulation Acceptance Testing: Ensures the software complies with regulatory requirements in industries like healthcare or finance.

Methodologies:

User Acceptance Testing (UAT): The most common form of acceptance testing, where actual users test the software to ensure it solves their problems.

Business Acceptance Testing (BAT): Conducted by business stakeholders to confirm that the system meets business requirements.

Operational Acceptance Testing (OAT): Focuses on operational aspects such as backups, deployment, and system monitoring.

Key Steps in Acceptance Testing:

Define acceptance criteria based on business needs and requirements.

Prepare test cases that reflect real-world use and scenarios.

Have end users or stakeholders execute the tests in a controlled environment.

Collect feedback and validate the software’s alignment with user needs.

Review results and resolve any identified issues.

Sign-off from the client or business stakeholders to approve the system for release.

Differences Between System Testing and Acceptance Testing:

Aspect	System Testing	Acceptance Testing
Objective	Ensure the software meets all functional and non-functional requirements.	Ensure the software meets business needs and user requirements.
Performed By	Testers or QA engineers.	End users, business stakeholders, or client representatives.
Scope	Focuses on the entire system.	Focuses on real-world use cases and business objectives.
Type of Test	Both functional and non-functional tests.	Primarily functional and user-experience tests.
Environment	Controlled test environment.	Real-world or production-like environment.

Acceptance Criteria

1. Acceptance criteria-Product acceptance
    - During the requirements phase, each requirement is associated with acceptance criteria. It is possible that one or more requirements may be mapped to form acceptance criteria (for example, all high priority requirements should pass 100%). Whenever there are changes to requirements, the acceptance criteria are accordingly modified and maintained.

2. Acceptance criteria—Procedure acceptance

- Acceptance criteria can be defined based on the procedures followed for delivery. An example of procedure acceptance could be documentation and release media. Some examples of acceptance criteria of this nature are as follows.

  1. User, administration and troubleshooting documentation should be part of the release.

  2. Along with binary code, the source code of the product with build scripts to be delivered in a CD.

  3. A minimum of 20 employees are trained on the product usage prior to deployment.

- These procedural acceptance criteria are verified/tested as part of acceptance testing.

3. Acceptance criteria–Service level agreements

- Service level agreements (SLA) can become part of acceptance criteria. Service level agreements are generally part of a contract signed by the customer and product organization. The important contract items are taken and verified as part of acceptance testing.
- For example, time limits to resolve those defects can be mentioned part of SLA such as:
    - All major defects that come up during first three months of deployment need to be fixed free of cost;
    - Downtime of the implemented system should be less than 0.1%;
    - All major defects are to be fixed within 48 hours of reporting.

With some criteria as above (except for downtime), it may look as though there is nothing to be tested or verified. But the idea of acceptance testing here is to ensure that the resources are available for meeting those SLAs.

Selecting Test Cases for Acceptance Testing-what test cases can be included for acceptance testing.

1. End-to-end functionality verification Test cases that include the end-to-end functionality of the product are taken up for acceptance testing. This ensures that all the business transactions are tested as a whole and those transactions are completed successfully. Real-life test scenarios are tested when the product is tested end-to-end.

2. Domain tests Since acceptance tests focus on business scenarios, the product domain tests are included. Test cases that reflect business domain knowledge are included.

3. User scenario tests Acceptance tests reflect the real-life user scenario verification. As a result, test cases that portray them are included.

4. Basic sanity tests Tests that verify the basic existing behavior of the product are included. These tests ensure that the system performs the basic operations that it was intended to do. Such tests may gain more attention when a product undergoes changes or modifications. It is necessary to verify that the existing behavior is retained without any breaks.

5. New functionality When the product undergoes modifications or changes, the acceptance test cases focus on verifying the new features.

6. A few non-functional tests Some non-functional tests are included and executed as part of acceptance testing to double-check that the non-functional aspects of the product meet the expectations.

7. Tests pertaining to legal obligations and service level agreements Tests that are written to check if the product complies with certain legal obligations and SLAs are included in the acceptance test criteria.

8. Acceptance test data Test cases that make use of customer real-life data are included for acceptance testing.

Executing Acceptance Tests

- An acceptance test team usually comprises members who are involved in the day-to-day activities of the product usage or are familiar with such scenarios.
- The product management, support, and consulting team, who have good knowledge of the customers, contribute to the acceptance testing definition and execution.
- They may not be familiar with the testing process or the technical aspect of the software. But they know whether the product does what it is intended to do. An acceptance test team may be formed with 90% of them possessing the required business process knowledge of the product and 10% being representatives of the technical testing team.
- The number of test team members needed to perform acceptance testing is very less, as the scope and effort involved in acceptance testing is not much when compared to other phases of testing.

- The role of the testing team members during and prior to acceptance test is crucial since they may constantly interact with the acceptance team members.
- Test team members help the acceptance members to get the required test data, select and identify test cases, and analyze the acceptance test results.
- During test execution, the acceptance test team reports its progress regularly. The defect reports are generated on a periodic basis.
- Defects reported during acceptance tests could be of different priorities. Test teams help acceptance test team report defects. Showstopper and high-priority defects are necessarily fixed before software is released.
- In case major defects are identified during acceptance testing, then there is a risk of missing the release date.
- When the defect fixes point to scope or requirement changes, then it may either result in the extension of the release date to include the feature in the current release or get postponed to subsequent releases.

NON-FUNCTIONAL TESTING

- Functional testing involves testing a product's functionality and features.
- Non-functional testing involves testing the product's quality factors.
- Non-functional testing is performed to verify the quality factors (such as reliability, scalability etc.). These quality factors are also called non-functional requirements.
- Non-functional testing requires the expected results to be documented in qualitative and quantifiable terms.
- Non-functional testing requires large amount of resources and the results are different for different configurations and resources.
- Non-functional testing is very complex due to the large amount of data that needs to be collected and analyzed.
- The focus on non-functional testing is to qualify the product and is not meant to be a defect-finding exercise. Test cases for non- functional testing include a clear pass/fail criteria.

- Non-functional tests results are also determined by the amount of effort involved in executing them and any problems faced during execution.
- For example, if a performance test met the pass/fail criteria after 10 iterations, then the experience is bad and test result cannot be taken as pass.
- Either the product or the non-functional testing process needs to be fixed here.
- Non-functional testing requires understanding the product behavior, design, and architecture and also knowing what the competition provides.
- It also requires analytical and statistical skills as the large amount of data generated requires careful analysis.
- Failures in non-functional testing affect the design and architecture much more than the product code.
- Since nonfunctional testing is not repetitive in nature and requires a stable product, it is performed in the system testing phase.

Objectives of Non-functional Testing

- **Increased usability:** To increase usability, efficiency, maintainability, and portability of the product.
- **Reduction in production risk:** To help in the reduction of production risk related to non-functional aspects of the product.
- **Cost Reduction:** To help in the reduction of costs related to non-functional aspects of the product.
- **Optimize installation:** To optimize the installation, execution, and monitoring way of the product.
- **Collect metrics:** To collect and produce measurements and metrics for internal research and development.
- **Enhance knowledge of product:** To improve and enhance knowledge of the product behavior and technologies in use.

Security Testing

Ensures that the system is protected against threats and unauthorized access.

Methodologies

- Vulnerability Testing: Identifies weaknesses in the system that could be exploited.
- Penetration Testing: Simulates an attack on the system to find vulnerabilities that could be used by hackers.
- Authentication and Authorization Testing: Verifies that users only have access to the resources they are authorized to access.
- Data Encryption and Integrity Testing: Ensures that sensitive data is encrypted and protected during storage and transmission.
- Session Management Testing: Checks if sessions are securely managed, ensuring users can't hijack or manipulate sessions.

Usability Testing

Assesses how user-friendly and intuitive the system is for its target audience.

Methodologies

- User Interface (UI) Testing: Evaluates whether the user interface is easy to navigate, intuitive, and aesthetically pleasing.
- User Experience (UX) Testing: Involves assessing the overall experience of the user when interacting with the system, ensuring it is smooth, efficient, and enjoyable.
- Accessibility Testing: Ensures the system is accessible to users with disabilities (e.g., screen reader compatibility, keyboard navigation).
- Consistency Testing: Ensures that the application maintains consistent design, terminology, and behaviors throughout.

Performance Testing

- Is a type of non-functional testing that focuses on assessing how well a software application performs under various conditions.
- The goal of performance testing is to identify performance bottlenecks, measure system behavior under stress, and ensure that the system can handle expected user loads and beyond.
- It ensures that the application meets performance requirements in terms of speed, scalability, reliability, and resource usage.